

IRCクライアント体験ハンズオン

所要時間: 約30分
対象: ft IRC プロジェクト参加検討者
形態: 2名以上で実施推奨

0. ft IRC 課題の目標

IRCサーバープログラム `ircserv` を作る！

- IRCクライアント（`irssi`など）から接続できる
- ユーザー登録（`PASS/NICK/USER`）ができる
- チャンネルに入って会話できる（`JOIN/PRIVMSG`）
- オペレーター機能（`KICK/INVITE/TOPIC/MODE`）が動く

1. IRCとは（3分）

Internet Relay Chat - 1988年生まれのテキストチャットプロトコル。

- Slack/Discordの先祖
- 1つのサーバーに複数クライアントが接続
- チャンネル（部屋）で会話する
- 今も現役で使われている（オープンソースコミュニティ等）

歴史

年代	出来事
1988	IRC誕生（フィンランド、Jarkko Oikarinen） ^[1]
1990s～2004	爆発的普及、ピーク時1000万人同時接続 ^[2]
2003～	SNS台頭で衰退開始（2012年までに60%減） ^[2:1]
2024現在	OSSコミュニティで健在（Libera Chat: 約3万人） ^[3]

Discordとの違い（重要）

Discordの「サーバー」とIRCの「サーバー」は意味が違う。

Discord	IRC
「○○のDiscordサーバー」＝コミュニティ/グループ	チャンネル（ <code>#channel</code> ）に相当
Discord社のインフラ（見えない）	IRCサーバー（ <code>ircserv</code> ）に相当 ← 今回作るもの

【Discord】
Discord社のサーバー（技術インフラ）← ユーザーは意識しない

└ 「○○ゲームのサーバー」← ユーザーが「サーバー」と呼ぶもの。ユーザーが接続先として意識する。

└ `#雑談`

└ `#攻略`

【IRC】
IRCサーバー（`ircserv`）← ユーザーが接続先として意識する。今回作るもの。

└ `#雑談`

└ `#攻略`

つまり: 今回作る `ircserv` は、Discord社が裏で動かしているサーバープログラムに相当する。

もう1つの大きな違い: 永続性

項目	Discord	IRC（ft IRC）
アカウント	永続（登録制）	存在しない ^[4]
ニックネーム	アカウントに紐づく	早い者勝ち、接続中のみ有効 ^[5]
チャンネル	永続（誰かが作れば残る）	誰もいなくなったら消える

→ IRCは「切断したら終わり」。次に接続したらまたnickを設定し、チャンネルに参加し直す。

公開IRCサーバー（誰でも接続可能）

Discordサーバーは招待制が多いが、IRCには誰でも接続できる公開サーバーがある。

サーバー	Server Address ^[6]	利用者
Libera Chat	irc.libera.chat ^[7]	Linux, Arch, KDE等のOSSコミュニティ（約32,000人）
OFTC	irc.oftc.net	Debian等（約15,000人）
freenode	irc.freenode.net ^[8]	衰退中（2021年の騒動で多くがLiberaへ移行）

今回のハンズオンでは **Libera Chat** に接続する。

2. irssiインストール（2分）

irssi^[9] はターミナルベースのIRCクライアントソフトウェア。今回のハンズオンではこれを使ってIRCサーバーに接続する。

```
brew install irssi
```

インストール確認:

```
irssi --version
```

3. 公開サーバーに接続（5分）

接続

`irc.libera.chat` は実在する公開IRCサーバー。今回作る `ircserv` と同じ役割のプログラムが動いている。

```
irssi -c irc.libera.chat -n 自分のニックネーム
```

(`-c`: 接続先サーバー、`-n`: ニックネーム指定) ^{[10][11]}

例:

```
irssi -c irc.libera.chat -n taro
```

注意: 進行の都合上ニックネームは参加者が `taro` `hanako` `torinoue` いずれかにする（被ると接続できない）

接続成功の確認

以下が表示されれば成功:

```

-!- End of /MOTD command.
-!- Mode change [+Ziw] for user taro
[(status)]
```

- `End of /MOTD command.` = サーバーからの案内表示完了
- `Mode change [+Ziw]` = ユーザー設定完了
- `[(status)]` = 入力待ち状態（画面最下部のプロンプト）

4. 基本操作（10分）

コマンド一覧（リファレンス）

irssiのコマンド。IRCプロトコルに準拠しており、ircservが処理する。

コマンド	動作	例
<code>/join #チャンネル名</code>	チャンネル参加	<code>/join #42test</code>
<code>/names #チャンネル名</code>	メンバー一覧	<code>/names #42test</code>
<code>/msg 相手 内容</code>	DM送信	<code>/msg taro hi!</code>
<code>/nick 新名前</code>	ニックネーム変更	<code>/nick jiro</code>
<code>/topic #チャンネル名</code>	トピック確認	<code>/topic #42test</code>
<code>/topic #チャンネル名 内容</code>	トピック設定	<code>/topic #42test Welcome!</code>
<code>/mode #チャンネル名 +i</code>	招待制に設定	<code>/mode #HOGEHOGE +i</code>
<code>/invite ユーザー #チャンネル名</code>	ユーザーを招待	<code>/invite hanako #HOGEHOGE</code>
<code>/part #チャンネル名</code>	チャンネル退出	<code>/part #42test</code>
<code>/quit</code>	切断	<code>/quit</code>

シナリオ: 3人でIRC体験（約12分）

役割を決めて進行する。2人の場合は誰かがtorinoueを兼任。

役割:

- **taro**: 最初にチャンネルを作る人（オペレーターになる）
- **hanako**: 後から参加する人
- **torinoue**: さらに後から参加する人

シナリオ1: 基本の会話（5分）

前提: 各自 `-n taro` , `-n hanako` , `-n torinoue` で接続済み

【taro】チャンネルを作成

```
/join #42test
```

→ taroが最初に入ったので、自動的にオペレーターになる

【hanako, torinoue】チャンネルに参加

```
/join #42test
```

【taro】メンバー確認

```
/names #42test
```

→ 結果: `[@taro] [ hanako] [ torinoue]`

→ `@taro` の `@` はオペレーターの印

【全員】チャンネルで会話

taro:

```
hello everyone!
```

hanako:

```
hi taro!
```

torinoue:

```
nice to meet you!
```

【hanako】taroにDMを送る

```
/msg taro Hi! coffee later?
```

→ taroにだけ届く、torinoueには見えない

【taro】DMを確認する

irssiでは受信DMは別ウィンドウに届く。下部に `[Act: 2]` のような表示が出たら:

```
Alt+2
```

または `Ctrl+N` で次のウィンドウへ遷移。`Alt+1` または `ESC->1` で `#42test` に戻る。

【taro】ニックネームを変更してみる

```
/nick jiro
```

→ チャンネル内で「taro is now known as jiro」と表示される

【hanako】メンバー確認

```
/names #42test
```

→ 結果: `@jiro hanako torinoue` (taroがjiroに変わった)

【jiro】元に戻す

```
/nick taro
```

シナリオ2: TOPICを設定（2分）

【taro】チャンネルのトピック(お題)を設定

```
実行前: irssi画面最上段のステータスバー（ブルー背景の抜き文字）に注目

/topic #42test This is the topic
```

→ 結果: ステータスバーに This is the topic が表示される

【hanako】トピックを確認

```
/topic #42test
```

→ 結果: メッセージ欄に Topic for #42test: This is the topic と表示される

【hanako】トピックを変更してみる

```
/topic #42test hanako wants to change
```

→ 結果: メッセージ欄に #42test You're not a channel operator と表示される（エラー）

→ オペレーター権限がないため変更できない

シナリオ3: 招待制チャンネル（5分）

【taro】新しい秘密のチャンネルを作成

```
/join #HOGEHOG
```

【taro】招待制に設定

```
/mode #HOGEHOG +i
```

【hanako】秘密のチャンネルに入ろうとする

```
/join #HOGEHOG
```

→ 結果: メッセージ欄に Cannot join to channel #HOGEHOG (You must be invited) と表示される（エラー）

→ 招待制（+i）のため、招待されていないユーザーは入れない

【taro】hanakoを招待する

```
/invite hanako #HOGEHOG
```

【hanako】招待されたので入れる

```
/join #HOGEHOG
```

→ 成功！

【torinoue】招待されていないので入れない

```
/join #HOGEHOG
```

→ 結果: メッセージ欄に Cannot join to channel #HOGEHOG (You must be invited) と表示される（エラー）

→ まだ招待されていないため入れない

【taro】招待制を解除する

```
/mode #HOGEHOG -i
```

【torinoue】入れるようになった

```
/join #HOGEHOG
```

→ 成功！（招待制が解除されたため）

【全員】秘密のチャンネルで会話

```
taro:

welcome to the secret room!
```

シナリオ終了

【全員】チャンネルを退出

```
/part #H0GEH0GE
/part #42test
```

【全員】接続終了

```
/quit
```

体験のまとめ: ircservで実装する機能【仕様】

下表の「機能」はIRCプロトコルのコマンド（ircservが処理する）。「irssiコマンド」は対応するクライアント側コマンド。

機能	根拠	体験	irssiコマンド	説明
PASS	明示	-	(自動送信)	サーバーパスワード認証
NICK	明示	✔	/nick , -n	ニックネーム（表示名、変更可）
USER	必須 ^[12]	-	(自動送信)	ユーザー名（識別子、変更不可）
JOIN	明示	✔	/join	チャンネル参加
PRIVMSG	明示	✔	/msg , 直接入力	メッセージ送信（チャンネル/DM）
PART	暗黙	✔	/part	チャンネル退出
QUIT	暗黙	✔	/quit	サーバーから切断
PING	必須	✔	(自動応答)	生存確認要求
PONG	必須	✔	(自動応答)	生存確認応答
TOPIC	明示	✔	/topic	トピック確認/設定
KICK	明示	-	/kick	ユーザーを追放
INVITE	明示	✔	/invite	ユーザーを招待
MODE +i	明示	✔	/mode +i	招待制チャンネル
MODE +t	明示	✔	/mode +t	TOPIC変更をオペレーター限定
MODE +k	明示	-	/mode +k	チャンネルパスワード
MODE +o	明示	-	/mode +o	オペレーター権限の付与/剥奪
MODE +l	明示	-	/mode +l	ユーザー数制限

凡例:

- 根拠「明示」= 課題書に明記
- 根拠「必須」= IRCプロトコル上必須（RFC 1459）
- 根拠「暗黙」= 課題書の「similar to any official IRC server」から必要（irssiが使用するため）
- 体験「✔」= 今日体験した機能
- 体験「✔」= 間接的に体験（Libera Chatのデフォルト設定）
- 体験「-」= 体験していないが実装が必要

これらの機能を全部、自分たちでC++98で実装する！

5. 「これをC++98で作る」（3分）

何を作るのか

さっき接続した irc.libera.chat は、どこかのサーバーで動いているプログラム。

今回の課題では、同じようなサーバープログラム ircserv をC++98で自作する。

【さっきやったこと】
irssi (クライアント) → irc.libera.chat (Libera Chatが運営するサーバー)

【課題で作るもの】
irssi (クライアント) → ircserv (私たちが作るサーバー) ← これを実装

なぜクライアントではなくサーバーを作るのか

クライアント（irssi）はオープンソースで公開されている^[13]ため、既存のものを使えばよい。

しかし本当の理由は、学べる技術がサーバー側に多いからと思われる^[14]:

技術要素	この課題で学ぶこと
I/O多重化（poll）	複数クライアントを1プロセスで同時に扱う
ノンブロッキングI/O	1つのクライアントに待たされずに処理を続ける
バッファリング	TCPストリームからメッセージ境界を正しく切り出す
並行処理の状態管理	複数クライアント/チャンネルの状態を一貫して管理

一言でまとめると: 並行ネットワークプログラミングの基礎技術

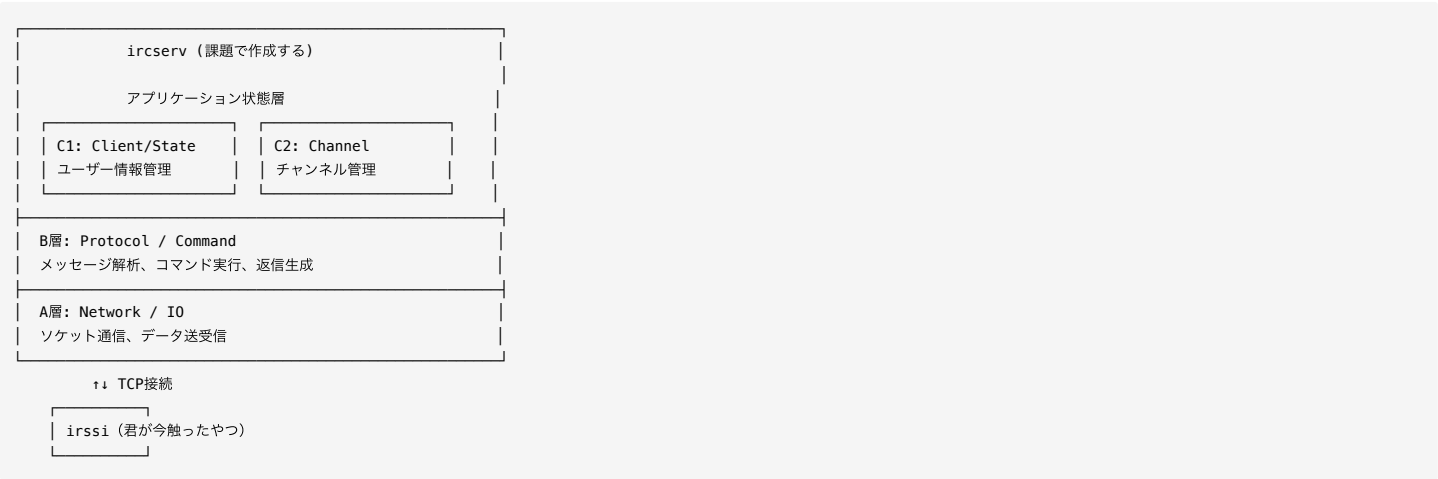
クライアント側だとこれらは「あれば便利」程度だが、サーバー側だと**必須**。だからサーバーを作る。

完成したら:

- 自分のPCで `./ircserv 4242 password` を起動
- irssiから `irssi -c localhost -n test_nick` で接続
- さっきやった操作（/nick, /join, /msg等）が全部動く

4層アーキテクチャ

ircservの内部構造:



さっきのコマンドと層の対応

操作	関わる層
接続した瞬間	A層（ソケット接続）
<code>/nick</code>	B層（解析）→ C1層（ユーザー情報更新）
<code>/join</code>	B層（解析）→ C2層（チャンネルにメンバー追加）
メッセージ送信	B層（解析）→ C2層（配送先特定）→ A層（送信）

6. ncでプロトコル覗き見（5分・オプション）

nc (netcat) = 生のTCP通信ができるツール。デバッグやコードレビューで使う。

公開サーバーに生で接続

```
nc irc.libera.chat 6667
```

※ポート番号については脚注^[15]を参照

IRCプロトコルを手打ち

```
NICK jiro
USER jiro 0 * :Test User
```

※ Real Name（:以降）は本名である必要はない。任意の文字列で、WHOIS コマンドで表示される。

サーバーから返信が来る:

```
:irc.libera.chat 001 jiro :Welcome to the Libera.Chat...
```

何が見えるか

- irssiが裏で送っていた生のテキスト
- コマンド パラメータ\r\n という形式

- これを自分で解析する部分がB層

生プロトコル体験シナリオ

taro, hanako, torinoue が #HOGEHOGE にいる状態で試す:

【jiro】チャンネルに参加

```
JOIN #HOGEHOGE
```

【jiro】チャンネルにメッセージ送信

```
PRIVMSG #HOGEHOGE :hello from nc!
```

【jiro】taro にDM送信

```
PRIVMSG taro :direct message from nc
```

【jiro】チャンネルのメンバー覧

```
NAMES #HOGEHOGE
```

【jiro】切断

```
QUIT :bye
```

PING timeout を体験する

何もせずに放置すると、サーバーから以下が届く:

```
PING :calcium.libera.chat
```

これに PONG :calcium.libera.chat を返さないと:

```
:jiro!~jiro@... QUIT :Ping timeout: 245 seconds
ERROR :Closing Link: ... (Ping timeout: 245 seconds)
```

→ 切断される。これが PING/PONG 実装が必須な理由。

補足: 42のコードレビューでレビュアーがncを使って動作確認することがある。

終わったら `Ctrl+C` で切断。

7. 担当選びのヒント（2分）

担当	向いている人
A層（Network/IO）	ネットワーク/低レイヤー好き、ソケット触りたい
B層（Protocol/Command）	パーサー/文字列処理好き、RFC読むの苦じゃない
C1層（Client/State）	状態管理/データ構造好き、辞書設計したい
C2層（Channel）	ルール/権限設計好き、モード管理に興味ある

興味がある層があれば、詳細なオンボーディング資料を読んでみよう。

8. 終了

```
/quit
```

お疲れ様！

次のステップ（アサイン後）

担当が決まったら、以下を読む:

担当	ファイル
A層	onboarding_A.md
B層	onboarding_B.md
C1層	onboarding_C1.md

担当	ファイル
C2層	onboarding_C2.md

参考資料

本ドキュメントで使したコマンド・情報の一次資料:

項目	資料
irssiソースコード	https://github.com/irssi/irssi
irssi manページ	https://man.archlinux.org/man/irssi.1
irssi公式ドキュメント	https://irssi.org/documentation/help/connect/
Libera Chat	https://libera.chat/
Libera Chat統計	https://netsplit.de/networks/statistics.php?net=libera.chat
IRC歴史（創設者記述）	https://www.mirc.com/history.html
IRC Wikipedia	https://en.wikipedia.org/wiki/Internet_Relay_Chat
IRCプロトコル	RFC 1459, RFC 2812

1. Jarkko Oikarinen "History of IRC" <https://www.mirc.com/history.html> — IRC創設者本人による記述。"The birthday of IRC was in August 1988." [↩](#)
2. Wikipedia "Internet Relay Chat" https://en.wikipedia.org/wiki/Internet_Relay_Chat — "IRC reached 6 million simultaneous users in 2001 and 10 million users in 2004–2005" / "losing around 60% of users between 2003 and 2012" [↩](#) [↩](#)
3. netsplit.de libera.chat統計 <https://netsplit.de/networks/statistics.php?net=libera.chat> および Libera Chat Annual Report 2024 <https://libera.chat/annual-reports/2024/> [↩](#)
4. **アカウントについて** — Libera.Chat等の大規模IRCネットワークでは「NickServ」という登録サービスでアカウント機能を提供するが、**ft_ircでは実装不要**。 [↩](#)
5. **ニックネームについて** — irssiは `~/irssi/config` にニックネームを保存するため、次回起動時に同じnickで接続を試みる。
これはクライアント側の機能であり、サーバー側の永続化ではない。詳細フロー:

1. irssi起動
 - ~/irssi/config から nick="foo" を読む

2. サーバーに接続
 - NICK foo を送信

3. サーバー側チェック
 - "foo" 使用中? → NO → "使っていいよ" (001 RPL_WELCOME)
 - "foo" 使用中? → YES → "ダメ" (433 ERR_NICKNAMEINUSE)

4. /quit
 - サーバー側: Client削除、nick "foo" 解放
 - irssi側: ~/irssi/config に "foo" 保存済み

5. 次回接続時
 - また "foo" で試みる (たまたま空いていれば成功)
- 重要:** サーバーはnickを永続化しない。「前回と同じnickで接続できた」のは、irssiが設定を覚えていて、かつそのnickがたまたま空いていたから。 [↩](#)
6. `irssi -c Server Address -n 自分のニックネーム` のようにサーバーに接続するために使う。HTMLで書かれたブラウザで見るためのページのドメイン名ではない。
ちなみに、URL (Uniform Resource Locator) は「通信手段（プロトコル）＋場所」なので `https://www.oftc.net/` は (https:// というWebの通信方法で、oftcのHPを開く)。「場所」と誤魔化したがるが厳密にはFQDN (Fully Qualified Domain Name : 完全修飾ドメイン名) である。FQDN = ホスト名 (www) + ドメイン名 ([oftc.net](https://www.oftc.net/)) [↩](#)
7. irssi内部に"LiberaChat":"irc.libera.chat"という辞書があるらしく `irssi -c LiberaChat -n 自分のニックネーム` **でも繋がる** [↩](#)
8. irssiで接続後、不安定なら `/connect -tls -tls_verify irc.freenode.net 6697` で**証明書提示**すれば入れるみたい。 [↩](#)
9. irssi — ターミナルベースのIRCクライアント。
Wikipedia <https://ja.wikipedia.org/wiki/Irssi> [↩](#)
10. irssi(1) manページ <https://man.archlinux.org/man/irssi.1> — `-c` (サーバー指定) および `-n` (ニックネーム指定) オプションの一次資料 [↩](#)
11. irssi公式ドキュメント <https://irssi.org/documentation/help/connect/> — /CONNECTコマンドの詳細 [↩](#)
12. USERコマンドはRFC 1459で接続時に必須。課題書の "a username" は曖昧だが、irssiは接続時に `USER <username> 0 * :<realname>` を自動送信するため、ircservはこれを**受信・解析**する必要がある。サーバーがOSユーザー名を取得するのではなく、クライアントが送信する文字列を処理するだけ。 [↩](#)
13. irssi (クライアント) GitHub <https://github.com/irssi/irssi> [↩](#)
14. サーバー側もオープンソースで公開されている。
Solanum (C言語, 40MB) : <https://github.com/solanum-ircd/solanum>、InspIRCd (C++, 54MiB) : <https://github.com/inspircd/inspircd> 等。
ただしInspIRCdはC++17相当 (GCC 7+/Clang 5+) であり、ft_ircのC++98とは互換性がない。
また両者とも大規模ネットワーク向けで、ft_ircの参考にするには規模が大きすぎる。
参考にすべきは**課題書に添付のbircd** (学習用、シンプル、select()使用)。 [↩](#)
15. **Libera.Chat のポート番号** — 6667: 平文 (非SSL)、6697: SSL/TLS。Libera.Chat は平文接続を制限している場合がある。反応がなければ SSL が必要かもしれない (nc では SSL 接続は難しい。代替として `openssl s_client -connect irc.libera.chat:6697` または `ncat --ssl irc.libera.chat 6697` を使用)。 [↩](#)